

Optimisation des Architectures des Réseaux de Neurones (NAS)

Projet 8INF976

Romain AMIGON

Avril 2026

Les limites du Deep Learning actuel :

- La conception des réseaux repose massivement sur l'empirisme (l'expérience du développeur).
- Tendance à concevoir des modèles surdimensionnés (obèses) par rapport à la tâche réelle.

L'objectif du projet :

- Automatiser et formaliser mathématiquement la recherche d'architectures (Neural Architecture Search - NAS).
- Comparer différentes méthodes d'optimisation pour trouver le réseau optimal.

Approche classique : On définit manuellement l'architecture puis on optimise les poids W .

$$g : \mathbb{R}^n \rightarrow \mathcal{F}$$

Approche NAS : On ajoute un niveau d'abstraction. On cherche l'architecture optimale θ^* (topologie et hyperparamètres) qui génère la meilleure fonction g .

$$f : \Theta \rightarrow \mathcal{F}$$

$$f(\theta) = g_\theta$$

un forward dans un réseau de neurone est: $f(\theta)(W)(X) = y$, avec θ l'architecture du réseau, optimisée avec mes méthodes, W les poids, optimisé par entraînement, et X les données d'entrée, y la sortie.

- **Reinforcement Learning (2017)** : Approche fondatrice, efficace mais extrêmement coûteuse (22 400 jours-GPU).
- **One-Shot NAS & DARTS (2018-2019)** : Espace de recherche continu et partage de poids pour accélérer la recherche (1,5 jour-GPU).
- **Zero-Cost Proxies (2021)** : Évaluation d'un réseau sans entraînement via l'analyse des gradients à $t = 0$ (moins de 3 secondes).
- **LLMs & Transformers (GPT-NAS, 2023+)** : Apprentissage de la "grammaire" des réseaux pour générer des architectures optimisées.

Métaheuristiques :

- **Recuit Simulé (SA)** : Optimisation stochastique, rapide mais erratique.
- **Algorithme Génétique (GA)** : Évolution par tournoi et croisements.
- **Artificial Bee Colony (ABC)** : Intelligence en essaim avec mécanisme d'abandon pour éviter les minimums locaux.

Contrôleurs Auto-Régressifs (RL) :

- **Agent LSTM** : Modèle séquentiel classique entraîné par Policy Gradient.
- **Agent Transformer** : Innovation du projet. Intègre une entropie dynamique pour forcer l'exploration en cas de stagnation.

- `layer_classes.py` : Définition stricte et modulaire des couches (Conv2dCfg, LinearCfg, etc.).
- `model.py` (**DynamicNet**) : Moteur de traduction. Convertit les configurations en un graphe PyTorch évaluable, en calculant dynamiquement les dimensions (ex: lien CNN-Flatten-Dense).
- `optimizer.py` :
 - Fonction `evaluate` : Proxy d'évaluation avec Early Stopping local.
 - Opérateur de mutation (`neighbor`) : altère la topologie tout en garantissant la viabilité mathématique du graphe.

Titre de ta slide

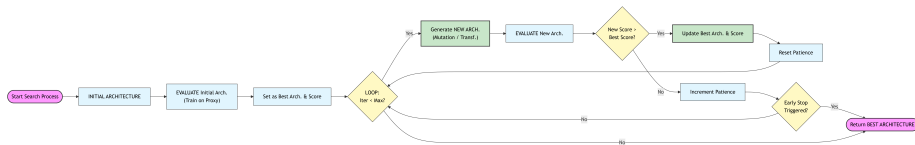
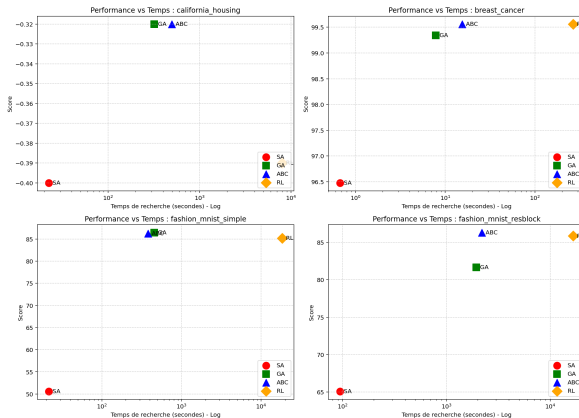


Figure: CF Github c'est mieux

Données Tabulaires et Vision Basique (CPU) :

- **California Housing** : MSE de -0.32 (ABC). Compétitif avec des architectures créées manuellement.
- **Breast Cancer** : Précision de 99.56% (ABC). Le plafond technique du dataset est atteint.
- **Fashion-MNIST (Bridé à 3% des données)** : 86.85% avec le Transformer. Prouve une excellente capacité d'extraction de features avec très peu de données.



Performances du Transformer :

- **California Housing** : -0.36 (Max: -0.31) — Temps : 2358s
- **Breast Cancer** : 98.90% (Max: 100%) — Temps : 56s
- **Fashion MNIST (Simple)** : $86.85 \pm 1.29\%$ — Temps : 1302s
- **Fashion MNIST (ResBlock)** : $85.85 \pm 1.56\%$ — Temps : 1494s

Contexte : Dataset extrêmement déséquilibré (0.2% de fraude). L'Accuracy n'est plus une métrique viable.

Adaptation du NAS :

- Modification dynamique de `evaluate` pour optimiser le **F1-Score**.
- Processus autonome : Génération par Transformer, micro-optimisation par ABC.

Résultats (Sur GPU) :

- Rappel de 84% et Précision de 71%.
- F1-Score de **0.77** sur la classe minoritaire, dépassant les MLPs classiques et compétitif face aux modèles d'arbres (XGBoost).

Recherche sur 50% du dataset puis train sur 100.

Comparaison des stratégies sur CIFAR-10 :

- **Recuit Simulé (100 itér.)** : Instable ($73.90\% \pm 3.46\%$).
- **ABC Seul (30 itér.)** : Dépendant de l'initialisation (79.76%).
- **Hybride Mémétique (Transformer 20 itér. + ABC 15 itér.)** : Score de **83.48%**. Le Transformer trouve rapidement une excellente ossature (Warm-Start), affinée ensuite par l'ABC. $\sim 5h$

Recherche et train sur 100% du dataset.

Stress-Test CIFAR-100 : Plafond à 52.23%. Confirme que le framework trouve l'optimum, mais reste limité par la simplicité de son espace de recherche actuel (absence de ResBlocks complexes ou d'Attention). $\sim 6h$

Bilan :

- Formalisation réussie : L'approche hybride automatisée (Transformer + ABC) surpasse la conception empirique.
- NAS Frugal : Architectures trouvées en quelques heures sur un GPU grand public (vs milliers de jours-GPU).
- Modularité totale par l'utilisateur (ex: F1-Score pour la fraude).

Perspectives :

- **Zero-Cost Proxies** : Remplacer l'entraînement partiel par des métriques purement mathématiques.
- **Enrichissement du Search Space** : Intégrer des cellules Inception ou Inverted Residuals pour les tâches complexes.
- **Front de Pareto** : Optimisation multi-objectifs stricte (Précision vs Latence).